Spatially Cohesive Service Discovery and Dynamic Service Handover for Distributed IoT Environments

Kyeong-Deok Baek and In-Young Ko(✉)

School of Computing, Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea
{kyeongdeok.baek, iko}@kaist.ac.kr

Abstract. The proliferation of the Internet of Things (IoT) enables the provision of diverse services that utilize IoT resources distributed in ad-hoc network environments. This has resulted in a new challenge, the issue of how to efficiently and dynamically discover appropriate IoT services that are necessary to accomplish a user task in the vicinity of the user. In this paper, we propose a service discovery method that finds IoT services from a user's surrounding environment in a spatially cohesive manner so that the interactions among the services can be efficiently carried out, and the outcome of service coordination can be effectively delivered to the user. In addition, to ensure a certain Quality of user Experience (QoE) level for the user task, we develop a service handover approach that dynamically switches from one IoT resource to an alternative one to provide services in a stable manner when the degradation of the spatial cohesiveness of the services is monitored. The spatio-cohesive service discovery and dynamic service handover algorithms are evaluated by simulating a mobile ad-hoc network (MANET) based IoT environment. Then, various service discovery strategies are implemented on this simulation environment, and several options for the service discovery and handover algorithms are tested. The simulation results show that compared to various baseline approaches, the proposed approach results in a significant improvement in the spatial cohesiveness of the services discovered for user tasks. The results also show that the approach efficiently adapts to dynamically changing distributed IoT environments.

Keywords: Internet of things · Distributed service discovery · Service handover · Spatio-cohesive service coordination · Task-oriented computing

1 Introduction

Recent years have witnessed a wide spread proliferation of Internet of Things (IoT) and this has [1] enabled the provision and deployment of diverse services that utilize IoT resources in urban environments. For effective collection of data and management of IoT services, recent commercial IoT-based systems, such as Microsoft Azure IoT Hub[1],

[1] https://azure.microsoft.com/en-us/services/iot-hub/.

J. Cabot et al. (Eds.): ICWE 2017, LNCS 10360, pp. 60–78, 2017.
DOI: 10.1007/978-3-319-60131-1_4

have adopted the cloud computing model. However, owing to a rapid increase in the number of IoT resources [2] in recent years, a centralized cloud computing model no longer remains scalable. Therefore, recent studies have suggested distributed computing models based on the Mobile Ad-hoc Network (MANET) [3] to address the scalability issue [4]. In this distributed model, as can be seen in Fig. 1, IoT resources are accessible directly via a MANET, which allows for IoT resources to interact with each other in order to provide services to users in a more efficient and flexible manner without the need to deploy infrastructure. In addition, composite services can be developed by integrating IoT and other types of services such as Web and cloud services together to accomplish user tasks.

For user-centric provision of IoT services, we developed a task-based service provision framework in our previous research [5]. In this framework, a *task* is a description of a user's demand, which entails the services required to provide necessary functionalities. A *service* defines a set of functionalities that are necessary to support an activity for a user task. A service is realized through a service instance that requires a set of *resources*, which are IoT devices that provide certain primitive functionalities. In this paper, we assume that IoT resources have enough computational power and networking capabilities to form a MANET without any infrastructure. However, even if an IoT resource is constrained by limited computation or networking capabilities, a *service gateway* may be used to enable the resource to join a MANET.

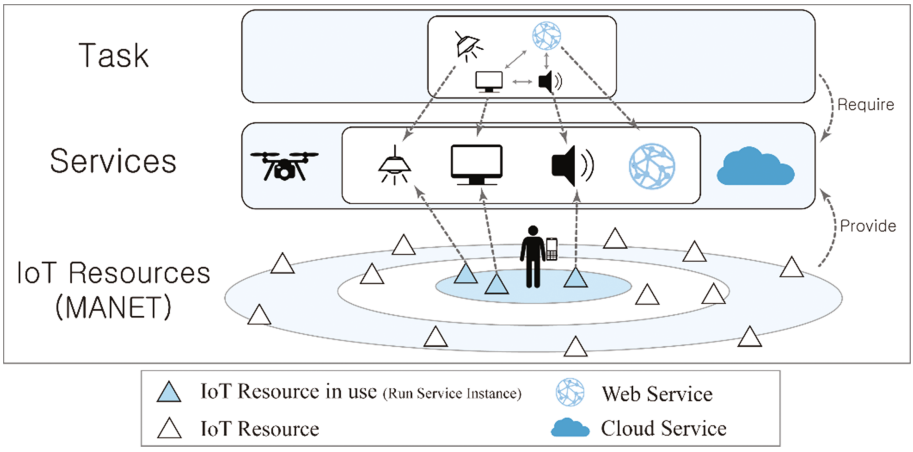

Fig. 1. Distributed IoT Environment Using MANET

Such a distributed IoT environment, however, raises a new challenge to efficiently and dynamically discovering appropriate IoT resources, which provides necessary IoT services to accomplish a user task, in the vicinity of the user. To address this issue, several studies have been undertaken in the service-oriented architecture and Web

services domains [6]. In particular, mobility, location dependency, and composability of distributed services have been considered as important issues to address to enable effective service provision in practical IoT environments [7]. First, owing to mobility of users and IoT resources, the availability of IoT services frequently changes. Therefore, a discovery algorithm need to dynamically discover valid IoT services that are necessary to perform user tasks. Second, unlike traditional Web services, the spatial location of a user and IoT resources affects the effectiveness of delivering IoT services to users. This implies that, as many studies point out [6, 7], to deliver the outputs of IoT services to a user in an effective and efficient manner, corresponding IoT resources need to be located close to the user. Third, various services in an IoT environment need to coordinate with each other to perform complex user tasks, and therefore, cooperative IoT resources need to be located cohesively in a space where the user is located.

Even though there have been many studies on evaluating locational context of IoT services [6], none of them have focused on ensuring IoT resources that are involved with service coordination to be spatially cohesive to each other, to accomplish user tasks effectively. In this paper, we define *spatio-cohesive services* as services that utilize IoT resources that are located cohesively in a user's vicinity with the distance among the services short enough such that the outcome of the service coordination can be effectively delivered to the user. The spatio-cohesiveness of IoT services is a critical factor to maximize the Quality of user Experience (QoE) of a user task. Therefore, when degradation of spatio-cohesiveness of IoT services is detected, some of the services need to be replaced with alternative ones in the MANET so that a certain degree of spatio-cohesiveness that is required by a user task can be ensured.

Consequently, in this paper, we propose a spatio-cohesive service discovery and dynamic service handover approach for ad-hoc IoT environments, where spatio-cohesiveness of IoT services is essential to perform a user task with high QoE. The proposed approach consists of two phases. In the first phase, a service discovery plan is generated, which is a systematic plan for discovering services that can be coordinated together to accomplish a user task. In this phase, various service discovery strategies such as a spanning tree algorithm and a service priority function are considered to generate the most effective discovery plan. In the second phase, spatio-cohesive services are discovered and handed over using the service discovery plan generated in the first phase. The spatio-cohesive service discovery algorithm first discovers a core set of services that meet the spatio-cohesiveness requirements of a task for the user, and then incrementally discovers other services while ensuring that the services are spatially cohesive with the services that have already been discovered. In addition, the dynamic service handover algorithm monitors the spatio-cohesiveness status of the services and performs handover when some of the services are beyond the requirement boundary.

We evaluated the spatio-cohesive service discovery and handover algorithms by simulating a distributed IoT environment. Further, we implemented the service discovery strategies on the simulation environment and tested two options for allowing concurrent service discovery and discovering multiple candidate services with the service discovery plans. Because there is no research on spatio-cohesiveness among

services, we implemented baseline as a basic algorithm that does not consider spatial distance among services. Consequently, we observed that the proposed algorithms show improvements in terms of spatio-cohesiveness of services discovered for user tasks.

The remainder of this paper is organized as follows. Section 2 discusses related work on service discovery in dynamic environments. Section 3 explains the spatio-cohesive service discovery and dynamic service handover method. Section 4 outlines the evaluation conducted of the proposed approach and analyzes the simulation results obtained. Section 5 introduces our testbed implementation and demo scenario. Finally, Sect. 6 concludes this paper.

2 Related Work

Service discovery in service computing environments can be classified into two types: proactive and reactive service discovery [8]. Reactive service discovery algorithms discover services according to a user's request, while proactive service discovery algorithms discover services before a user's request based on advertisement from IoT resources. In distributed IoT environments, it is necessary to discover services in a reactive manner according to the requirements of user tasks because of scalability. Furthermore, in IoT environments, a service discovery often requires the discovery of IoT resources that are necessary to provide the service [8]. Therefore, service discovery in this paper is closely associated with the discovery of the required IoT resources. In particular, the service discovery process in IoT environments includes the selection of the best IoT resources among candidate resources that can provide required services.

There have been various studies on scalable discovery of required services for performing user tasks in MANET environments [8, 9]. A MANET is an infrastructure-less environment, which can enable resources with high mobility to be connected with each other in a scalable manner. However, owing to the infrastructure-less characteristics of MANET, we cannot maintain a centralized service registry to discover services. Therefore, service discovery in a MANET needs to be performed in a decentralized manner [8]. Although this makes service discovery in a MANET more complex than a centralized environment, it enables heterogeneous and mobile IoT services to be discovered in a robust manner.

To improve the efficiency of service discovery in a MANET, some studies use a cross-layer method that merges service discovery into network capabilities [8, 10]. Their work focuses on optimizing network protocols for service discovery rather than improving QoE that users can observe.

One recent study points out that it is important to coordinate IoT services based on a goal-oriented framework to effectively deal with complex demands from users [4]. In particular, they state that it is essential to find multiple services that need to participate in service coordination to accomplish a user goal. However, in a dynamic MANET environment, where the availability of services changes often, it is challenging to discover multiple services within a user's vicinity. Furthermore, providing failure-resilient

service discovery is also a challenging issue owing to the highly dynamic nature of MANET [4], and diverse contexts of services to consider [6, 11].

With regard to delivering services to users while ensuring a high QoE, context-aware service discovery has been considered as an important issue in service provision [6, 11]. In these studies, contextual information such as a user's preferences and location are utilized to increase the integrated quality of discovered services from users' perspectives and in terms of the efficiency of network architecture. In particular, the location of services is a critical context that affects QoE in performing user tasks [12]. For example, in a recent study, the goal was to optimize the integrated quality of discovered services by considering the relative locations of the services within a network [13]. In this research, they use a genetic algorithm to improve the quality of discovered services in an iterative manner. However, this approach requires frequent verification of the status of the network, and therefore it may incur considerable communication effort among services that are distributed over a dynamic network environment like MANET.

3 Spatially Cohesive Service Discovery and Dynamic Service Handover

Figure 2 shows the main phases and overall process of the spatio-cohesive service discovery and dynamic service handover algorithms. In the first phase, the spatio-cohesiveness requirements are extracted and a plan for the service discovery and handover is generated. A *spatio-cohesiveness requirement* is a locational dependency between two services or between a user and a service, which should be located cohesively to each other. After the extraction, the requirements are represented in a graph, and the service discovery plan is instantiated on the graph, using various discovery strategies that guides traversal on the graph. In the second phase, IoT services for a user task in an IoT environment are discovered in a spatially cohesive manner based on the service discovery plan generated. The dynamic service handover step is

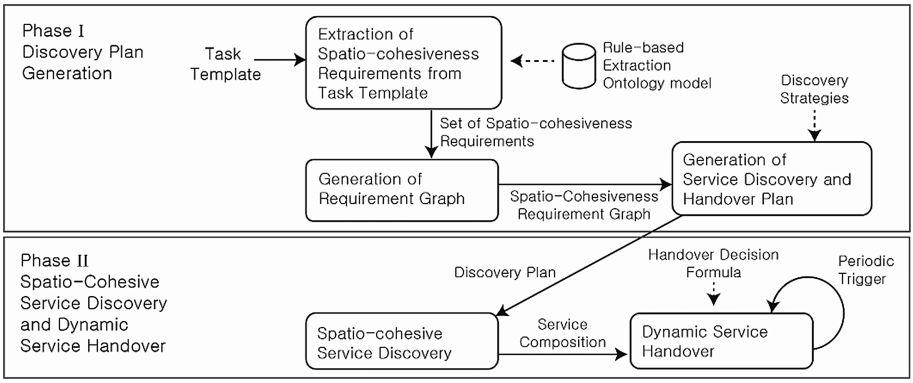

Fig. 2. The spatio-cohesive service discovery and handover process

periodically activated to monitor any changes in the spatio-cohesiveness requirements of the services, and to dynamically hand over some of the service instances that are not spatially cohesive to other services into alternative ones.

3.1 Problem Definition

The spatio-cohesive service discovery and dynamic service handover problem that this work tries to solve is defined formally as follows. For a given task, T, the problem is to find a proper set of IoT resources, which are necessary to provide a set of required services, S_T, of the task. Each service, s $\in S_T$, has its own service description, des_s, that contains the information about the required capabilities and constraints. A time series, TS, which is a sequence of time instances, τ_i, indicates a topology change of the MANET. Time instance τ_0 represents the time when the task starts its execution.

The MANET at time τ is represented as a graph, $N(\tau) = (D \cup \{u\}, E(\tau))$. The vertices of the graph include the set of IoT resources, D, which provide capabilities for services; and a user, u, that consumes the services. Each vertex v has function $v.\text{coord}(\tau)$ that returns the physical coordinate of the node at time τ. A service that can be provided by utilizing an IoT resource, $d \in D$, is represented as $d_{service}$. An edge set of the network at time, τ, is defined as $E(\tau)$, where $E(\tau)$ is a set of direct links that forms MANET among IoT resources and user. The elements of $E(\tau)$ change as the connections among the IoT resources and user change because of the mobility.

In this work, we assume that there is only one user consuming the services in the environment. Because the focus of this work is on service discovery for user tasks in a spatially cohesive manner, the issues of simultaneous service discovery and the resource allocation problem with multiple users will be covered in future work.

A *spatio-cohesiveness requirement* is defined as (m, n, l), where m and n are a service or a user (i.e., m, n $\in S_T \cup \{u\}$), and they need to be located spatially close to each other within an upper limit, l. R_T is a set of spatio-cohesiveness requirements for a task, T, that can be automatically extracted from the service descriptions in the task template. In addition, spatio-cohesiveness requirements can be extracted from a user preference or can be manually provided by a user (Table 1).

Table 1. Summary of Formal Notations

Name	Notation
Service set for task T	S_T
Time instance	$\tau_i \in TS$
Network	$N(\tau) = (D \cup \{u\}, E(\tau))$
Vertex position	$v.\text{coord}(\tau), v \in D \cup \{u\}$
Capability of IoT resource d	$d_{service}$
User	u
Connection	$(d_i, d_j)\ or\ (u, d_i) \in E(\tau)$
Spatio-cohesiveness requirement	$r_i = (m, n, l) \in R_T, for\ m, n \in S_T \cup \{u\}$
Candidate resource selection	$C(\tau) \subseteq 2^D$
Spatio-cohesive resource selection	$c_{SC}(\tau) \in C(\tau)$

The problem is to find the spatio-cohesive solution $c_{SC}(\tau) \in C(\tau)$, where $C(\tau) \subseteq 2^D$ is a set of candidate IoT resources that provide the services of the task at time τ. The spatial distance between the resources that provide the services in each service pair (services m and n), or between a resource that provides a service, m, and the user, n, of set c is defined as function $distance_c(m, n)$, and needs to be minimized for each spatio-cohesiveness requirement of the task. Moreover, achievement of the requirement is the ratio of distance and limit value, and will be 0 if distance exceed the limit. Thus, measurement of the achievement of a spatio-cohesiveness requirement, $r_i(\mathrm{c})$, with a candidate IoT resource set, c, can be measured as follows:

$$r_i(\mathrm{c}) = \max\left(1 - distance_c(r_i.m.coord(\tau), r_i.n.coord(\tau))/_{r_i.l}, 0\right) \tag{1}$$

In addition, according to the definition of the connectivity of a network [14], the spatio-cohesiveness of a candidate set of IoT resources can be measured as the average of the minimum distance of a service that causes failure to one of its spatio-cohesiveness requirements. Therefore, the spatio-cohesiveness objective function, $R_T(c)$, of a candidate resource set, c, can be defined as follows:

$$R_T(\mathrm{c}) = \sum\nolimits_{s \in S_T, u} \min\nolimits_{r_i=(s,s_k,l) \in R_T} (r_i(\mathrm{c}))/|S_T \cup \{u\}| \tag{2}$$

The spatio-cohesiveness of a candidate IoT resource set is a value between zero and one. A low value indicates that there are many failures of meeting the requirements by IoT resources, while a high value indicates that most of the spatio-cohesiveness requirements are strongly achieved by the resources.

The spatio-cohesive service discovery problem is to find a set of resources that provide the spatio-cohesive services at the initial time, i.e., $c_{SC}(\tau_0)$, and the dynamic service handover problem is to find a new set of resources that provide spatio-cohesive services when there is a change in the IoT MANET, i.e., $c_{SC}(\tau_i)(i > 0)$. Furthermore, the dynamic service handover problem has an additional objective function, $handover(\mathrm{c}(\tau_{i-1}), c(\tau_i))$, which is for identifying the number of handovers from $\mathrm{c}(\tau_{i-1})$ to $\mathrm{c}(\tau_i)$. The objective function for service handover can be defined as follows:

$$handover(\mathrm{c}(\tau_{i-1}), c(\tau_i)) = |\mathrm{c}(\tau_i) - c(\tau_{i-1})| \tag{3}$$

By minimizing the number of IoT resources to be handed over, we can reduce the overhead of service handovers in accordance with the changes in the IoT MANET. Therefore, the following defines the problem of finding spatio-cohesive services for a user task, while minimizing service handover cost throughout the lifecycle of the task:

$$c_{SC}(\tau_i) = \left\{ \begin{array}{rr} \min_{c \in C(\tau_i)}(R_T(c)) & (i = 0) \\ \min_{c \in C(\tau_i)}(R_T(c), handover(\mathrm{c}(\tau_i) - c_{SC}(\tau_{i-1}))) & (i > 0) \end{array} \right\} \tag{4}$$

Because there are multiple objective functions for each spatio-cohesiveness requirement with some constraints, this problem is a constrained multi-objective optimization problem. To check if the objective functions included in $R_T(c)$ are

conflicting with each other, let us consider an IoT resource, d_i, which is selected for the task. The discovery algorithm search for IoT resources that are necessary to provide the services that are spatially cohesive to the services of d_i. If we assume that all the resources and the user are distributed over the MANET randomly and uniformly, the resources are expected to be distributed in all directions from d_i. Therefore, choosing an alternative resource d_i' instead of d_i will make one of the spatio-cohesiveness requirement achievements decrease, while making another requirement achievement increase.

3.2 Discovery Plan Generation

Spatio-Cohesiveness Requirement Extraction. Studies on representing the functionality and the QoS of Web services semantically [12, 15] show that it is possible to infer the QoS requirements of a service in a semantic manner based on the semantic descriptions of the service. Similarly, the spatio-cohesiveness requirements of a user task can be extracted automatically from a task template. Therefore, as shown in Fig. 3, we define a brief ontology model to represent the spatio-cohesiveness requirements of a user task. The ontology model is developed using Web Ontology Language (OWL) and represented as a Unified Modeling Language (UML) class diagram. As shown in Fig. 3, a Task has its own *Task Template*, which is composed of multiple *Services*. A service can have some *Service Properties* representing the quality attributes and constraints of the service. In addition, a service can be classified into one or more *Service Category*. The services of a task and the *User* of the task can be combined to form a *Service Boundary*, by which a spatio-cohesiveness requirement can be specified for the task. Finally, a set of *Spatio-cohesiveness Requirements* are extracted from task template.

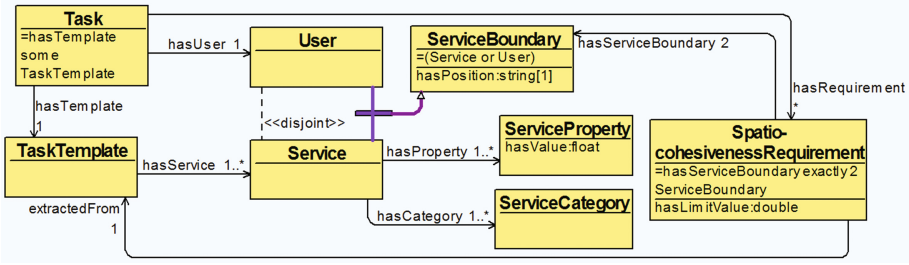

Fig. 3. Ontology model of spatio-cohesiveness requirements

Spatio-cohesiveness requirements that are represented based on this model can be extracted using a rule-based reasoning method. Figure 4 shows an example rule represented in Semantic Web Rule Language (SWRL). This rule states that if a task requires an audio service with volume property less than 30 dB, the distance between the user and the audio service should be less than 5 m:

```
Task(?t) ^hasTemplate(?t, ?tmpl)
^hasUser(?t, ?u) ^hasService(?tmpl, ?s)
^hasCategory(?s, Audio)
^hasProperty(?s, Volume)
^swrlb:lessThan(?v, 30)
```

⟶

```
hasRequirement(?t, ?req)
^hasServiceBoundary(?req, ?u)
^hasServiceBoundary(?req, ?s)
^hasLimitValue(?req, 5)
```

Fig. 4. SWRL rule for extracting spatio-cohesiveness requirements

Spatio-cohesiveness Requirement Graph Generation. In our approach, we represent the spatio-cohesiveness requirements as a graph called a *spatio-cohesiveness requirement graph (SCRG)*, in which the user and services are represented as vertices ($S_T \cup \{u\}$), and the spatio-cohesiveness requirements among them are represented as edges (R_T). We define the vertex that represents a user as the root of a SCRG. Figure 5 shows an example in which a SCRG is generated. The label on each edge represents the upper limit of spatio-cohesiveness requirement. The requirement graph is an abstraction of spatial positions and relationship of services. It does not represent the functional dependencies among services. In this paper, we assume that the user and all services in a SCRG are connected.

Discovery Plan Generation. On the basis of the SCRG generated in the previous stage, we now make a service discovery plan to discover services in a systematic manner. A service discovery plan for a task, T, is defined as follows:

$$P_T = (child_P, nonChild_P),$$

where $child_P$ is a subset of the spatio-cohesiveness requirements, and indicates the path in the requirement graph to discover a set of child services from its parent service. $nonChild_P$ is a complementary set of $child_P$. Therefore, the IoT resource that is selected for a service, s, is evaluated as to whether it meets the spatio-cohesiveness requirements of $nonChild_P$. Then, in accordance with the $child_P$, the child services for the service, s, are discovered in vicinity of the selected resource of s. Figure 5 shows an example of generation of a service discovery plan from a SCRG.

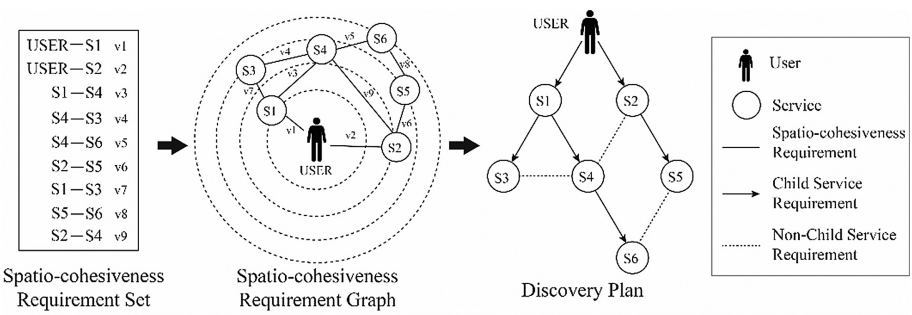

Fig. 5. Example of building a service discovery plan

Generation of the $child_P$ set is done by constructing a spanning tree of the SCRG. Therefore, our service discovery strategy is composed of a spanning tree algorithm and a service priority function. The spanning tree algorithm is for constructing the $child_P$, and the service priority function is for assigning a priority to each service based on their importance. This allows an algorithm to discover high-priority services effectively among child services of a service. Various spanning tree algorithms, such as Depth-First Search (DFS), Breadth-First Search (BFS), and Minimum Spanning Tree (MST) (e.g., the Prim's algorithm), can be used. For the service priority function, we can use an algorithm to calculate the centrality of a service in the requirement graph, which is a well-known indicator of a vertex's importance in the graph, or a requirement boundary from a service node to its parent service for strict prioritization.

3.3 Spatio-Cohesive Service Discovery and Handover

Spatio-Cohesive Service Discovery. Based on the service discovery plan built in the previous stage, the spatio-cohesive service discovery algorithm finds the IoT resources necessary to realize the services of a user task from the child services of the user node. Then, discovered services recursively finds the resources for the child services. For each resource, service discovery agent is deployed that allows discovery of child services in vicinity of the resource, as user requests. By the service discovery agent, user requests the resource of discovered service to discover its child services in vicinity of parent service. Therefore, service discovery request is propagated from parent service to child service, according to discovery plan.

During the service discovery process, for a service, there may be no available resource for its child service that meets the spatio-cohesiveness requirement. In this case, the algorithm re-discovers the resource of the parent service recursively until the service discovery of the parent service to its child service is successfully performed.

In this algorithm, there are two options that enable concurrent discovery of services, and the finding of multiple candidate resources for a service. Using these options, it is possible to make an efficient tradeoff between the failure rate and service discovery latency according to the conditions of the environment such as the dynamicity level, and the density of the deployed services.

Algorithm 1 shows the spatio-cohesive service discovery algorithm that is executed for each discovered service recursively. The service discovery is initially triggered by a user, and propagated to IoT resources. The inputs to the algorithm are a task, T, which includes own template, and a set of services, S_T, that need to be discovered for the task. A service discovery plan that is generated for the task is extracted from the task template, and the non-child services of the service to check the spatio-cohesiveness requirement are found based on the plan (Lines 2–3). For each non-child service, if it is already discovered and does not meet the requirement, the resource tries to discover the service of itself again from its parent (Lines 4–9). For simplicity, during the rediscovery of a service, current resource of the service is disabled, and an alternative resource is discovered (Lines 29–34). This makes the failed resource invisible in the further process.

If all the spatio-cohesiveness requirements of the non-child services are met, service discovery agent of the resource starts discovering child services of its associated service (Lines 10–16). Discovery of a child service is achieved by finding a set of candidate resources that can realize the child service in the vicinity of its parent service (Line 20). After a set of candidate resources is discovered, the resource selects the closest resource that can provide the child service, and starts discovery process from selected resource (Lines 23–26). If no available candidate resource is found for a child service, the resource tries to discover the parent service again (Lines 21–22). In addition, if the option is not allowed to discover the child services concurrently, it waits until the previous discovery session finishes, then proceeds to discovery of the next (Lines 13–15).

Algorithm 1. Spatio-cohesive Service Discovery Algorithm

```
 1: procedure ServiceDiscoveryTrigger(task T, service S)
 2:     nonChildSet = T.discoveryPlan.nonChildOf(S)
 3:     for each service nonChild in nonChildSet do
 4:         if isServiceDiscovered(nonChild) and not isRequirementMeet(S, nonChild) then
 5:             ReDiscovery(T, S)
 6:             Return
 7:         end if
 8:     end for
 9:     childList = T.discoveryPlan.childOf(S)
10:     for each service child in childList do
11:         ServiceDiscovery(T, S, child)
12:         if not allowConcurrency then
13:             waitUntilFinished(child)
14:         end if
15:     end for
16: end procedure
17:
18: procedure ServiceDiscovery(task T, service parent, service child)
19:     candidateSet = findCandidateSet(T, isRequirementMeet, parent, child)
20:     if isEmpty(candidateSet) then
21:         ReDiscovery(T, parent)
22:     else
23:         child.setResource(candidateSet.extractMin())
24:         ServiceDiscoveryTrigger(T, child)
25:     end if
26: end procedure
27:
28: procedure ReDiscovery(task T, service S)
29:     disableResource(S.getResource())
30:     T.resetDiscoveryState(S)
31:     parent = T.discoveryPlan.findParentOfService(S)
32:     ServiceDiscovery(T, parent, S)
33: end procedure
```

Algorithm 2 is for finding a set of candidate IoT resources by service discovery agent of the resource. The corresponding resource broadcasts the request of the child service (Line 3), then it finds candidates based on the replies from other

resources (Lines 4–10). If a replied resource can be used to provide a required child service and meets the spatio-cohesiveness requirement, the service discovery agent adds the resource to the candidate set (Lines 4–7). If the option is not allowed to find multiple candidate resources, it returns the candidate resource that is found first (Lines 8–10).

Algorithm 2. Algorithm to Find Candidate IoT Resources for a Service

```
1: procedure FindCandidateSet(task T, function condition, service parent, service child)
2:     candidateSet = emptySet
3:     parent.broadcastDiscoveryPacket(child)
4:     while time < delay and candidate = getDiscoveryResponse() do
5:         if condition(T, parent, candidate) then
6:             candidateSet.insert(candidate)
7:         end if
8:         if not allowMultipleCandidates then
9:             break
10:        end if
11:    end while
12:    Return candidateSet
13: end procedure
```

Dynamic Service Handover. The major difference between the traditional handover and the service handover in an IoT environment is that the service handover requires consideration of a set of services to perform a user task, while the traditional handover considers only a single access point. Moreover, it is necessary to consider not only the case of handing over between a user's mobile device and a resource, but also the case of handing over between resources. In our approach, we utilize the service discovery plan generated for a user task to perform service handover in a dynamic manner.

Algorithm 3 shows the dynamic service handover algorithm that has a structure similar to that of the spatio-cohesive service discovery algorithm. The service handover is triggered by a user periodically to check whether the resources selected for a task meet the spatio-cohesiveness requirements, and propagated to services similar with the service discovery algorithm. When handover process is triggered at a service by its parent service, associated service discovery agent on the resource finds a set of candidate resources for its child services by using a handover decision rule with hysteresis margin and threshold (Lines 27–34). The handover decision rule calculates the spatio-cohesiveness between resource of the parent service and the candidate resource, and between resource of the parent service and the current resource, according to the spatio-cohesiveness requirement of the parent service and the child service. Unlike the service discovery algorithm, empty candidate set indicates that there is no better resource available for the child service in the vicinity of the parent service.

Algorithm 3. Dynamic Service Handover Algorithm

```
1: procedure ServiceHandoverTrigger(task T, service S)
2:     nonChildSet = T.discoveryPlan.nonChildOf(S)
3:     for each service nonChild in nonChildSet do
4:         if not isRequirementMeet(S, nonChild) then
5:             ServiceHandover(T, S)
6:         end if
7:     end for
8:     childList = T.discoveryPlan.childOf(S)
9:     for each service child in childList do
10:        ServiceHandover(T, child)
11:        if not allowConcurrency then
12:            waitUntilHandoverEnds(child)
13:        end if
14:    end for
15: end procedure
16:
17: procedure ServiceHandover(task T, service S)
18:    parent = findParentOfService(S)
19:    candidateSet = findCandidateSet(T, handoverDecision, parent, S)
20:    if not isEmpty(candidateSet) then
21:        S.setResource(candidateSet.extractMin())
22:    end if
23:    ServiceHandoverTrigger(T, S)
24: end procedure
25:
26: procedure HandoverDecision(task T, service S, resource candidate)
27:    parent = T.discoveryPlan.findParentOfService(S)
28:    req = T.findRequirement(parent, S)
29:    currentResource = S.getCurrentResource()
30:    currentObj = getDistance(parent, currentResource)/req.getLimit()
31:    candidateObj = getDistance(parent, candidate)/req.getLimit()
32:    Return currentObj > threshold and currentObj − margin > candidateObj
33: end procedure
```

4 Evaluation

4.1 Evaluation Setting

We performed a number of simulations using the NS3 simulator[2] to evaluate the spatio-cohesive service discovery and dynamic handover algorithms. The NS3 simulator is one of the most well-known simulators for network environments. It provides various modules to simulate IoT environments that we exploited in this work. The simulations were conducted on a 64-bit Ubuntu 14.04 LTS operating system running on the virtual machine by Oracle VirtualBox 5.0.20 on Windows 10 Pro with an Intel i7-3770 processor. The size of the memory allocated for a virtual machine was 4 GB. In the simulation environment, 200 resource nodes were deployed in a 100 m × 100 m rectangular area in a uniform manner. Thus, each resource covered a 7 m × 7 m area,

[2] https://www.nsnam.org.

which is reasonable to deploying and managing resources in an IoT environment. Moreover, approximately half of the resources were generated as mobile nodes using the RandomDirection2d mobility model. We selected this model because it can reflect the behavior of mobile resources that move around in an IoT environment in an effective manner. Static resources were generated based on the ConstantPosition mobility model, which produces fixed position. The resource nodes communicated with each other through IEEE 802.11ac Wi-Fi channels with the VhtMcs0 physical mode, the most commonly used mode. In addition, for the Wi-Fi channels, we used the constant-speed propagation delay model, and the Cost-Hata propagation loss model, which are designed to simulate urban environments [16].

The number of services for a user task was set to 10 in order to simulate the situation of performing user task with medium complexity [4]. In addition, the velocity of the mobile resources was randomly set in the range 1 m/s to 10 m/s, which represents the walking and running speed of pedestrians in an IoT environment. For each service discovery strategy, we repeated the simulation 50 times, 20 s each. The simulation time set up for the experiments is much shorter than the time that usually takes to perform a user task in an IoT environment. However, in our simulation environment, 20 s were enough to test various situations that require dynamic service handovers. Before the simulation was initiated, the set of spatio-cohesiveness requirements, the SCRG, and service discovery plan of the simulation were generated based on a discovery strategy. At the beginning of each simulation, the spatio-cohesive service discovery is triggered by the user node. Once the service discovery process was completed, a series of service handover events were triggered one by one after a short delay.

4.2 Evaluation Results

Figure 6 shows the measurement of the spatio-cohesiveness metric over time. We tested the effectiveness of using the BFS and DFS strategies and compared their results against the case of discovering services without using the handover algorithm, and the case of not using a discovery plan. Without using a discovery plan, the service discovery process needed to find all of the services by the user node. Therefore, the spatio-cohesiveness requirements between a pair of services cannot be met, and this results in a poor spatio-cohesiveness condition. In the case of discovering services without using the handover algorithm, the result shows a continuous decrease in the spatio-cohesiveness of the services as time passes because of the mobility of resources. In contrast, the proposed BFS and DFS based approaches that use the service handover algorithm show high spatio-cohesiveness results under the condition of having mobile resources. As shown in Fig. 6, the BFS based approach and the DFS based approach do not have much difference.

Figures 7 and 8(a) compares the cases of using the options to allow concurrent service discovery and to discover multiple candidate services for the spatio-cohesive service discovery and dynamic service handover algorithms. Each box shown in the figure represents the simulation result generated by combining a service discovery strategy and an option. In the figure, the boxes show the number of service handovers

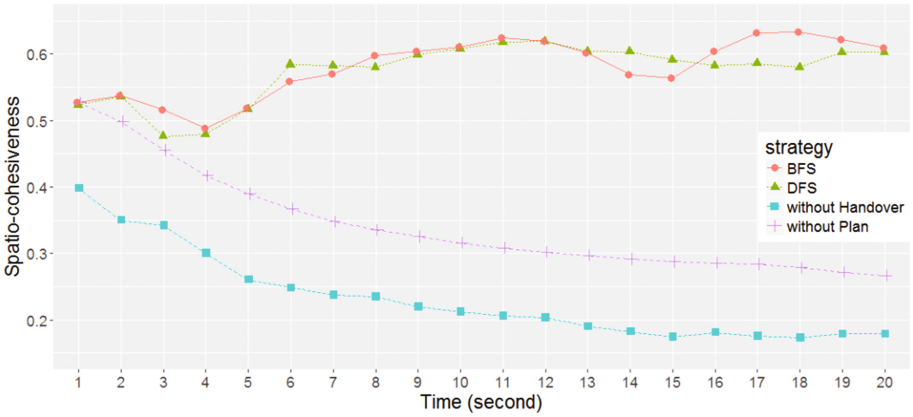

Fig. 6. Spatio-cohesiveness results compared to baseline approaches

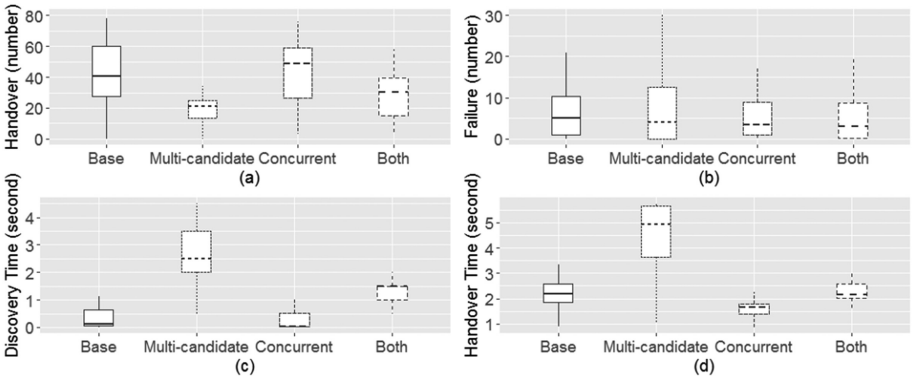

Fig. 7. Number of handovers (a), and number of failures (b), discovery time (c) and handover time (d) of using different combinations of service discovery strategies and options

made during the simulation (a), and the number of failures to meet spatio-cohesiveness requirements (b), and the time spent for the service discovery (c), and the time spent for the service handover (d), for each algorithm option.

The results show that choosing the option to allow multiple service discoveries effectively reduces the number of service handovers. These results are tightly related to the stability of the service discovery and handover algorithms. Considering multiple candidates during the process increases the stability of the algorithms (a less number of handovers means a more stable set of services), but makes them more time-consuming.

Choosing the option to allow concurrent service discovery resulted in the service discovery and handover time being less than that of the case in which this option was disabled. However, allowing concurrent discovery did not contribute significantly to improvement of the stability of the algorithms. Especially, allowing concurrent discovery effectively reduces the service discovery and handover time to find multiple candidate services when both options are used together.

Figure 8(a) compares the cases of using the options in terms of the spatio-cohesiveness. As shown in the figure, the case of enabling both options result in higher spatio-cohesiveness than other cases. Therefore, if the time constraint of service discovery and handover is not too hard, using both options of the algorithms can improve the stability and spatio-cohesiveness of the results.

Figure 8(b) compares the results of using the different service priority functions discussed in Sect. 3. For this experiment, service discovery plans were generated using BFS and three different service priority functions—random, degree centrality in a SCRG, and the limit value of the spatio-cohesiveness requirement of a service and its parent service. As can be seen in the figure, use of the service priority functions helps to improve spatio-cohesiveness of the services discovered especially during the service handover process (after the first time instance). However, the spatio-cohesiveness results of using different service priority functions are not significantly different. The case in which the limit value of the spatio-cohesiveness requirement is used results more steady spatio-cohesiveness of the services.

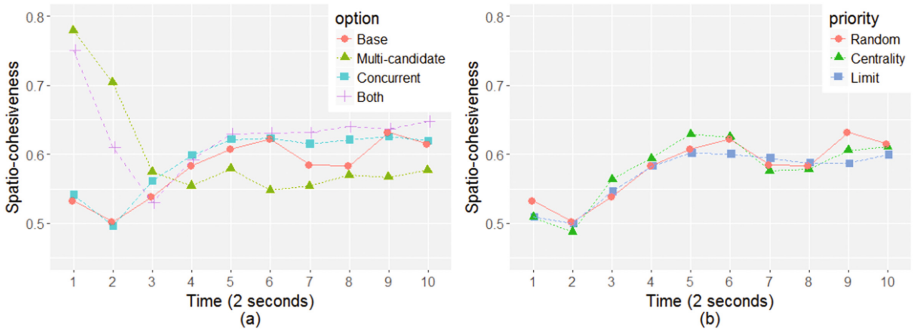

Fig. 8. Spatio-cohesiveness results of using different options (a), and different priority functions (b)

4.3 Threats to Validity

For the simulation, we did not consider the spatio-cohesiveness requirement extraction phase, and the simulation was conducted with randomly generated spatio-cohesiveness requirements. This was done because the spatio-cohesiveness requirement ontology model is not populated with practical task definitions yet. During the random generation process, we set the upper limit of requirements as 30 m to 45 m that covers about one-third of the entire environment area. Therefore, less than one-third of the IoT resources that are in the network are expected to spatially close within the upper limit. The lower bound of the limit seems high. However, because we consider mobile resources in dynamic IoT environments, such a setting might be appropriate.

5 Testbed Implementation

In this study, we implemented a testbed in which multiple IoT resources form a MANET. Because not many commercial IoT resources are programmable, we developed service gateways using Node-Red[3] on a Raspberry Pi 2 model B[4] that controls the power of IoT resources via Wemo[5] Insight model F7C029de. In addition, an Android application is developed to discover services by using the proposed algorithm as well as to perform user tasks on a mobile device (smartphone). In this testbed, once a user enters the MANET area, the user's mobile device automatically discovers the IoT resources that are necessary for performing the user task that the user selected from the available tasks in the testbed.

To measure the distance between resources, we use the Received Signal Strength Indicator (RSSI), which is commonly used for distance estimation. Therefore, while coordinating services for a task, if a user's mobile device detects degradation of the RSSI from an IoT resource that is used for providing a service, it discovers an alternative resource that meets the spatio-cohesiveness requirement of the task.

Figure 9 shows photos of the service-handover demonstration that we performed in our testbed. There are three IoT resources, Light 1 and 2, that can be used to provide a lighting service, and a fan, that can be used to provide a cooling service. In this environment, the user wants to perform a task, "Make a pleasant environment for reading." In Scene A, Light 1 and the fan are utilized to provide the necessary services for the user task. As the user moves away from Light 1 (Scene B), the system detects degradation in the spatio-cohesiveness between the lighting service the user. Therefore, in Scene C, the lighting service is handed over to Light 2, which is now closer to the user, and the QoE (appropriate lighting for reading) of the user task can be maintained. In this scenario, the cooling service is not handed over to an alternative IoT resource because the air from the

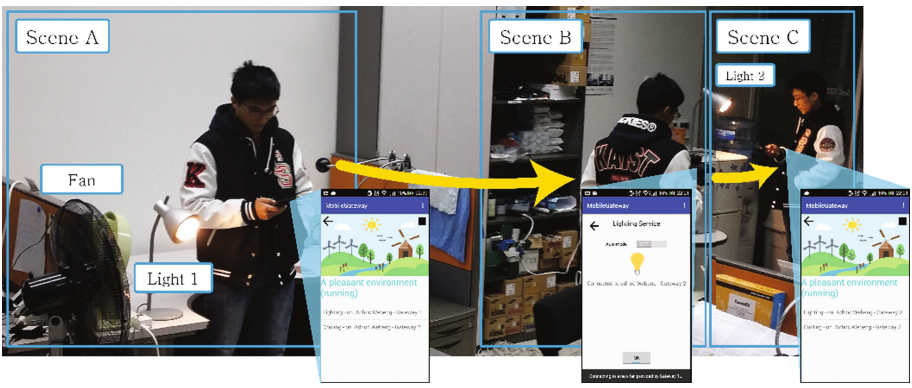

Fig. 9. Demonstration of a service-handover scenario in the testbed

[3] https://nodered.org/.

[4] https://www.raspberrypi.org/products/raspberry-pi-2-model-b/.

[5] http://www.wemo.com/.

fan can still reach the user in Scene C, and the QoE requirement (making the place cool) of continuously performing the user task can be met.

6 Conclusion

To perform a user task in an IoT environment, appropriate IoT services need to be discovered in a spatially cohesive manner in the vicinity of the user. Furthermore, to perform the user task in a stable manner under the highly dynamic MANET-based IoT environments, it is necessary to dynamically handover between services that utilize different IoT resources. In this paper, we proposed a service discovery method that finds IoT services from a user's surrounding environment in a spatially cohesive manner so that the interactions among the services can be effectively done, and the outcome of service coordination can be effectively delivered to the user. In addition, we developed a service handover approach to dynamically switching from an IoT resource to an alternative one to provide services in a stable manner when the degradation of the spatio-cohesiveness of the services is monitored.

The evaluation of the spatio-cohesive service discovery and dynamic service handover algorithms in an IoT environment was performed using the NS3 simulator. During the experiments, we observed that service discovery and handover using a discovery plan contributes to improving the spatio-cohesiveness level of the services discovered for user tasks. Furthermore, we found that enabling multiple candidate selections and concurrent service discovery are effective in improving the service discovery performance. In addition, the priority function used in building the discovery plans effectively increases the adaptability of the algorithms for dynamic IoT environments.

The main contribution of our work lies in its consideration of the spatial inter-relationship among services and user to discover effective services to improve user experience of performing a task, which is regarded as an important issue of mobile service discovery. Based on the spatial dependencies among the services, we organized the services to be discovered and handed over systematically, which improves efficiency and effectiveness of performing a user task. Moreover, our service handover algorithm reflects complex handover situations that are associated with the QoE of services.

In future work, we plan to extend our ontology model to represent diverse spatio-cohesiveness requirements, and QoE conditions. In addition, we will extend the service discovery and handover algorithms to deal with unconnected SCRG in order to support any type of service coordination for user tasks. Moreover, because the spatio-cohesiveness in service coordination should be considered as one of the critical factors of QoE, studies on the aspects of the spatio-cohesiveness requirements will be conducted.

Acknowledgment. This work was supported by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Science, ICT and Future Planning (2016R1A2B4007585).

References

1. Gubbi, J., et al.: Internet of things (IoT): a vision, architectural elements, and future directions. Future Gener. Comput. Syst. **29**(7), 1645–1660 (2013)
2. http://www.zdnet.com/article/internet-of-things-market-to-hit-7-1-trillion-by-2020-idc/
3. Corson, S., Macker, J.: Mobile ad hoc networking (MANET): routing protocol performance issues and evaluation considerations. No. RFC 2501 (1998)
4. Groba, C., Clarke, S.: Opportunistic service composition in dynamic ad hoc environments. IEEE Trans. Serv. Comput. **7**(4), 642–653 (2014)
5. Jimenez-Molina, A., Ko, I.-Y.: Spontaneous task composition in urban computing environments based on social, spatial, and temporal aspects. Eng. Appl. Artif. Intell. **24** (8), 1446–1460 (2011)
6. Truong, H.-L., Dustdar, S.: A survey on context-aware web service systems. Int. J. Web Inf. Syst. **5**(1), 5–31 (2009)
7. Elgazzar, K., Hassanein, H.S., Martin, P.: Daas: cloud-based mobile web service discovery. Pervasive Mob. Comput. **13**, 67–84 (2014)
8. Meshkova, E., et al.: A survey on resource discovery mechanisms, peer-to-peer and service discovery frameworks. Comput. Netw. **52**(11), 2097–2128 (2008)
9. Sailhan, F., Issarny, V.: Scalable service discovery for MANET. In: Third IEEE International Conference on Pervasive Computing and Communications. IEEE (2005)
10. Athanaileas, S.E., Ververidis, C.N., Polyzos, G.C.: Optimized service selection for manets using an aodv-based service discovery protocol. In: MEDHOCNET 2006, Corfu, Greece, June 2007
11. Surobhi, N.A., Jamalipour, A.: A context-aware M2 M-based middleware for service selection in mobile ad-hoc networks. IEEE Trans. Parallel Distrib. Syst. **25**(12), 3056–3065 (2014)
12. Chen, X., et al.: Web service recommendation via exploiting location and QoS information. IEEE Trans. Parallel Distrib. Syst. **25**(7), 1913–1924 (2014)
13. Klein, A., Ishikawa, F., Honiden, S.: Towards network-aware service composition in the cloud. Proceedings of the 21st International Conference on World Wide Web. ACM (2012)
14. Brandes, U., Erlebach, T.: Network Analysis: Methodological Foundations. LNCS, vol. 3418. Springer, Heidelberg (2005)
15. Ma, Q., et al.: A semantic QoS-aware discovery framework for web services. In: IEEE International Conference on Web Services, ICWS 2008 (2008)
16. Stoffers, M., Riley, G.: Comparing the NS-3 propagation models. In: 2012 IEEE 20th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems. IEEE (2012)